

Report for Cascade Chess Game Part A

Xiangdi (Allie) Hu (1524339)

Qianbin Dong (1511935)

Part 1: Explanation of the implementation of the search strategy

We used the A* search algorithm taught in the lecture. Our solution uses a two-phase graph search strategy. In Phase 1, we apply A* with evaluation function $f(n)=g(n)+h(n)$, where $g(n)$ is the path cost from the start state and $h(n)$ is the heuristic estimate of the remaining cost to a goal. Since all actions have the same cost, $g(n)$ is simply the number of actions taken so far. In Phase 2, only states with path length smaller than the current best solution length are explored by increasing g -value, so this phase is similar to BFS.

We use a priority queue (heap) as the frontier. Each entry stores the evaluation value, the current path length, a counter used as a tie-breaker, and the encoded board state. When we pop from the heap, the state with the smallest evaluation value is explored next. When we generate a legal successor that is new or reached by a shorter path, we push it into the heap. We also use a SearchContext object to cache useful information for each encoded state, including the board object, its heuristic value, its number of blue stacks, and its successor list. This reduces repeated computation. To reduce repeated exploration further, we apply symmetry reduction by treating identical, rotated, and mirrored board configurations as the same encoded state.

We define a state as the entire board configuration. To avoid duplicates, each board is encoded into a tuple. The key part of each heap entry is the evaluation function $f=g+h$.

In terms of implementation, we repeatedly pop a state from the heap and check whether it is a goal. If so, we stop exploring. Otherwise, we expand it using `get_new_possible_states`, which generates all legal successors according to the game rules. Not every generated successor is inserted into the heap. We first encode the successor, then compare its current path length with the best previously recorded path length for that encoded state. A successor is inserted only if it is new or reached by a strictly shorter path. This is handled by the `best_g` dictionary. We also store parent and parent_action so that the final action sequence can be reconstructed once a goal is found. If the heap becomes empty before reaching a goal, the board is treated as unsolvable.

For tie-breaking, when two states have the same evaluation value, we use an increasing counter so that the heap remains well-defined.

b = branching factor

d = optimal solution depth

m = maximum depth of the state space

U = upper bound (the best solution length found in Phase 1)

Time complexity

Phase 1: $O(b^d)$ typically, and $O(b^m)$ in the worst case

Phase 2: $O(b^U)$

Space complexity

Phase 1: $O(b^d)$ typically, and $O(b^m)$ in the worst case

Phase 2: $O(b^U)$

Part 2: Explanation of the heuristic

The heuristic function we build is:

h = the number of enemy (Blue) stacks on the board

+ distance_weight * the sum of the shortest distance per Blue stack

+ threat_weight * the sum of the greatest threat per Blue stack

We regard “the estimate of future steps to the goal” as “how we can eliminate the enemies as soon as possible”.

The base line is the number of enemy stacks on the board, since less enemies means we are closer to the goal.

Additionally, we include the distance and threat that is empowered by actions to measure “how soon”, shorter distance, greater threat imply we are approaching the goal faster, indicating we are on the shortest path.

Specifically, for each Blue stack, we find the largest threat and shortest distance with each Red stack, then sum them up to get the total threat and total distance (threat) that Blue stacks are currently suffering. Notice that

since we are adding the threat as a part of the heuristic function, small threat value actually means a greater threat in our case (though not very intuitive).

To make the heuristic more informed, we also adjust the threat weight, threat value and the distance weight through detecting patterns (flags) using an Enum class.

First, blue stacks are fortified. In this case, the cascade should be given more weight since pushing off is cheaper than eating, and if the cascade is not successful or meaningful we give direction more weight, and the distance should be given less weight since general closeness might be similar.

Second, pressures exist on the edges. In this case, Blue stacks are more vulnerable, so we give the threat more weight.

Third, Red stack scarcity occurs. In this case, we underline the largest threat all Blue stacks are undertaking (with fewer Red stacks, this would approach the goal faster) than the general closeness, so we give the threat more weight, and distance less (distance might be similar for some extreme cases).

Fourth, Blue stacks are scattered. In this case, the threat between different configurations would be similar (balanced) move to one, but away from the other, move to the other, away from the one. Hence, we should give distance more weight and threat less.

There are some limitations of the performance of our heuristic. Typically, our heuristic is not behaving decently regarding multiple edge and corner pressures and scattered, especially when there is a scarcity of Red stacks.

Since enemies are far from each other in this case and scattered around the board, the total distance, total threat would be similar during the process of moving the Red stack to one of the Blue stack, which is badly performed similarly even as BFS. Additionally, to maintain the admissibility, we only adjust the weights in a small range, therefore the adjustment phase might not be that influential. At the same time, the patterns we are detecting are quite limited and some of them are conflicting with each other.

Part 3: Discussion of the simultaneous case

Assumption: we assume that each Red stack can move in any direction and with all kinds of actions, and we can choose any number of Red stacks to move or not.

In our current implementation with only one action allowed at a time. For one node we have 16 possibilities.

Moreover, since we can only move one Red stack at the same time, the overall number of possible successors we are able to generate is at most the number of Red stacks times 16.

Considering the game with simultaneous actions allowed at a time, for one node: For one node we still have 16 possibilities. However, since we can move any number of Red stacks at the same time, the overall number of successors we are able to generate at most grows to $16 + 1$ (do not move, stay at the same position) n (the number of Red stacks)

Generally, we need more memory and more space, and we need to improve our heuristic function. Since our current heuristic is already considering all the distance and threat for each Blue stack with every Red stack, the formula of calculating the heuristic function could be a reasonable base. However, improvements are necessary. Due to the significant growth expected in the number of possible successors, we need to prune our searching graph with higher efficiency. Two essential aspects are as follows.

First, we need to add the logic of intelligent selection (pruning): We have to decide which Red stacks we would apply actions first to reach the goal more efficiently according to the current board configuration. For instance, we can try to name and detect more useful patterns that would help us to make the decision.

Second, we need to add the logic of checking conflict and ordering: Since now we are able to apply actions for more than one Red stacks, some of their actions might conflict with each other. For instance, two Red stacks might aim to eliminate one specific enemy, this is not meaningful and messy (even illegal). So I need to build up the logic for avoiding conflicting actions and the order/priority of actions among all possible moves of Red stacks on the board.

Meanwhile, since now we are able to reduce several enemies at the same time, we need to be more careful about maintaining the admissibility of the heuristic and not overestimate. It would be a reasonable method if we also relate the weight (in the heuristic function) with the number of enemies we are going to eliminate with the current board configuration.